Secure Checkout with RAG Payment Assistant

A card checkout that never charges twice, works for keyboard and screen-reader users, and explains failed payments with a cited help assistant.

Saumya Goswami · Technical documentation · October 2026

Live demo: 30-saumya-11.github.io/secure-checkout/ · GitHub: github.com/30-Saumya-11/secure-checkout

Contents

1. Overview

2. The problems it solves

3. Architecture

4. Never charging twice

5. Card rules

6. Accessibility

7. Payment help assistant (RAG)

8. Testing and CI/CD

9. Running it

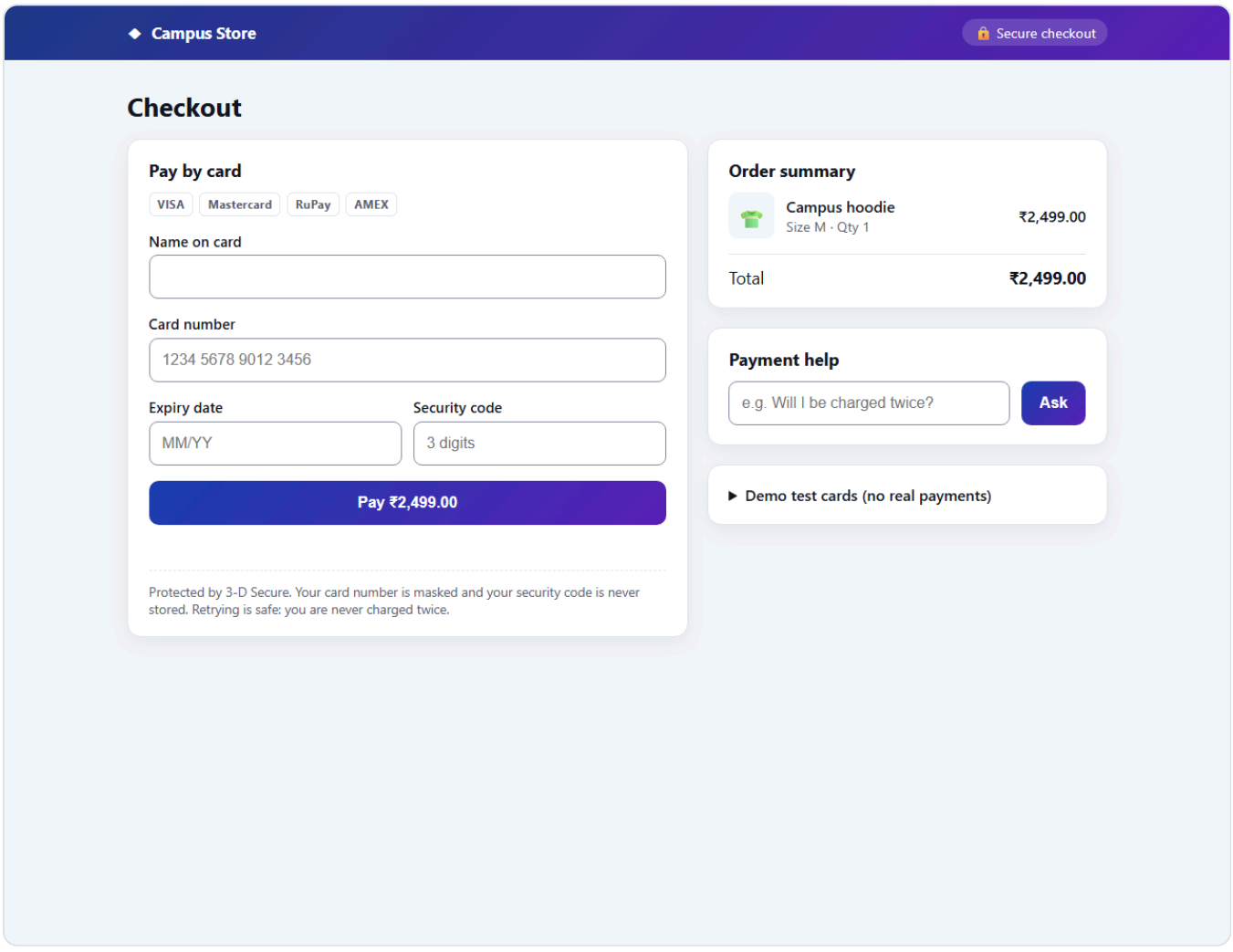
10. Limitations and next steps

11. Interview questions

1 Overview

Secure Checkout is a card payment page in **React + TypeScript** built around three promises to the customer: **you are never charged twice**, **everyone can use it** (keyboard and screen-reader users included), and **help is there when a payment fails**, from an assistant that answers from the help centre and cites its sources.

1 charge, even after double-clicks and lost responses	**0** automated accessibility violations (3 page states)	**94%** help-assistant recall@3 on held-out questions	**56** automated tests in CI

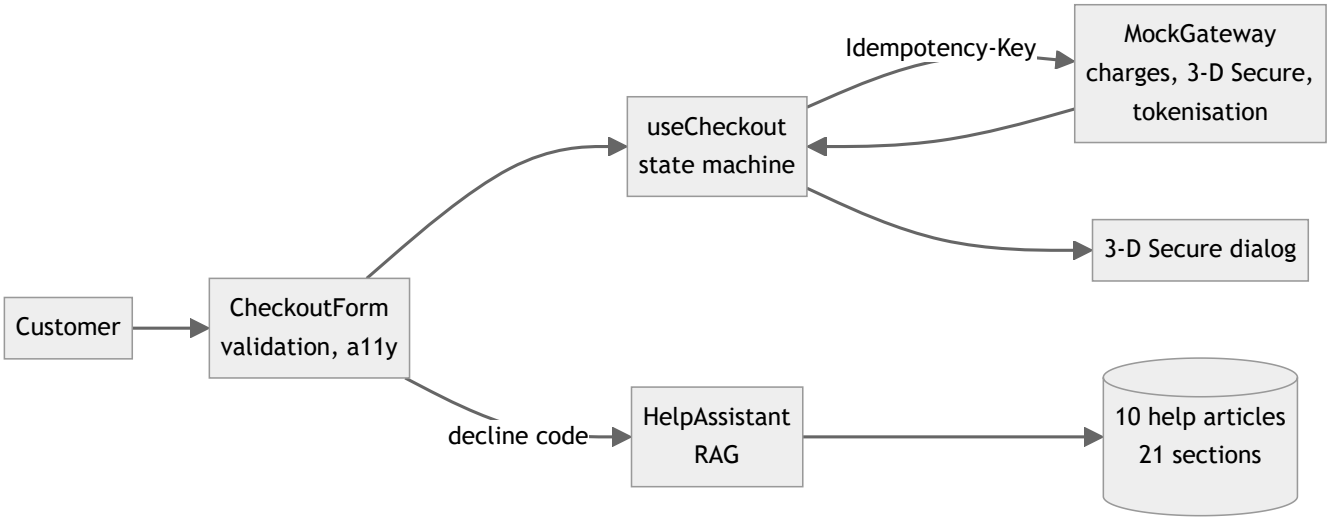


> Demo only: the payment gateway is a mock that runs in the browser and behaves like real gateways in the ways that matter to the UI. No real card network is contacted. Live: 30-saumya-11.github.io/secure-checkout

2 The problems it solves

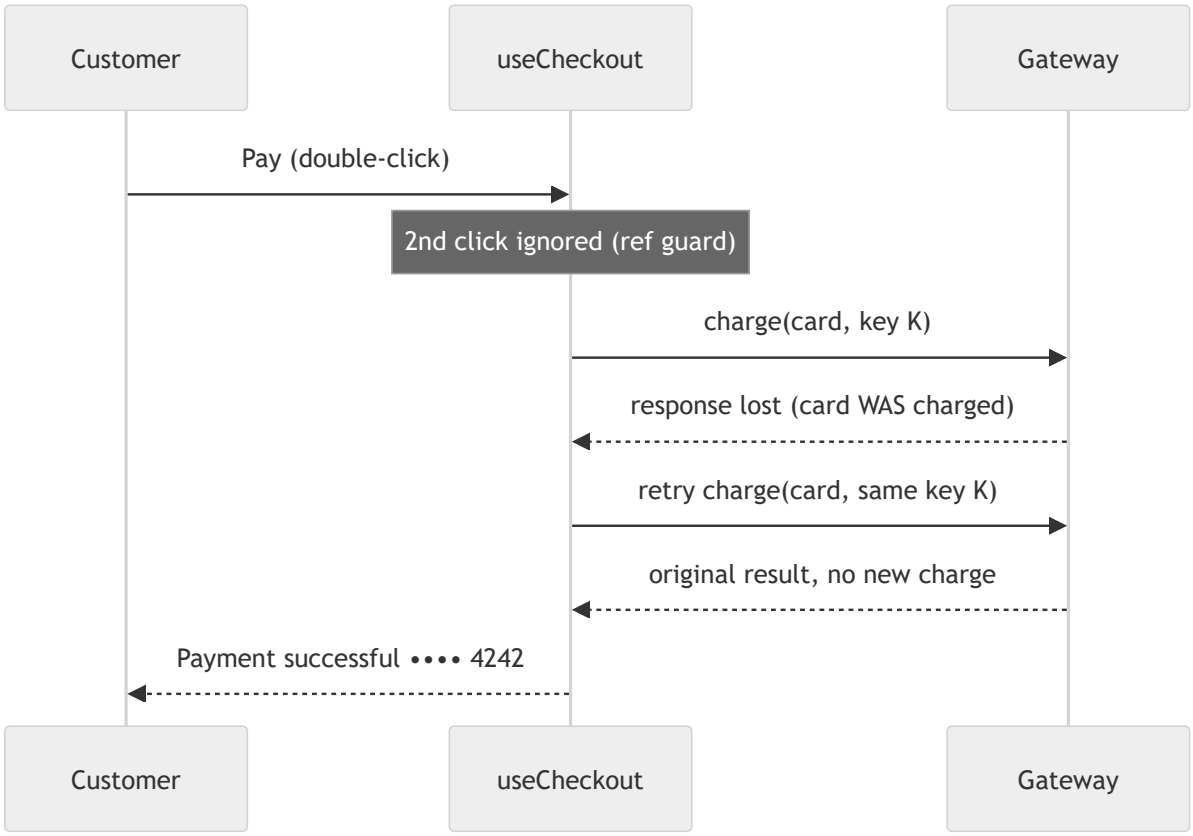
Problem	What goes wrong in real shops	What this checkout does
Double charges	The bank charges the card, the response is lost, the customer presses Pay again and is charged twice.	One idempotency key per attempt; retries reuse it, so the gateway returns the first result.
Inaccessible forms	Errors shown only in red, focus lost, no labels: screen-reader and keyboard users cannot pay.	Labelled fields, linked errors, an error summary that takes focus, live status updates, keyboard-only flow.
Card-data exposure	Full numbers or CVVs end up in logs or storage.	Only a token, the network and the last 4 digits are kept; the CVV is discarded and cleared after a failure.
Unexplained declines	"Payment failed" with no reason; customers give up or call support.	A help assistant that uses the actual decline code and cites the help article it used.

3 Architecture



File	Responsibility
`src/lib/card.ts`	Luhn checksum, network detection (IIN ranges), formatting, expiry and CVC rules, masking
`src/lib/gateway.ts`	Mock gateway: idempotency store, test-card declines, 3-D Secure, lost responses, tokenisation
`src/hooks/useCheckout.ts`	Checkout state machine and safe retry logic
`src/components/CheckoutForm.tsx`	Accessible form, error summary, live status
`src/components/ChallengeDialog.tsx`	One-time-code dialog with focus management
`src/assistant/*`	Help articles, BM25 retrieval, answering, evaluation questions

4 Never charging twice



- **One key per attempt.** The key is created when an attempt starts and reused for every retry of that attempt.
- **A new key only for a genuinely new attempt:** after a decline, or when the card details change.
- **Same key, different payment → refused** (`IdempotencyConflict`), so a bug cannot reuse a key by accident.
- **The double-click guard is a ref, not state:** a state update would only take effect on the next render, after the second click.
- Automatic retries (2, with back-off) after network errors; if they run out, the customer is told that pressing Pay again is safe.

5 Card rules

Network	IIN ranges used	Lengths	CVC
Visa	4	13, 16, 19	3
Mastercard	51–55, 2221–2720	16	3
RuPay	508500–508999, 606985–607984, 608001–608500, 652150–653149	16	3
American Express	34, 37	15	4

Luhn checksum: from the right, double every second digit (subtract 9 if it exceeds 9) and sum everything; a valid number sums to a multiple of 10. It catches every single-digit typo and most swaps of neighbouring

digits. Expiry: a card is valid until the end of its expiry month. Amounts are integers in paise, never floating point.

6 Accessibility

- Every input has a visible label and the right `autocomplete` token (`cc-number`, `cc-exp`, `cc-csc`, `cc-name`) and a numeric keyboard on phones.
- Errors appear after the user leaves a field (not while typing), are linked with `aria-describedby` and set `aria-invalid`.
- On a failed submit, focus moves to an error summary whose items link to the fields.
- Progress and results are announced through a polite live region.
- The 3-D Secure dialog takes focus and closes on Escape; touch targets are at least 44 px; contrast ratios are chosen for WCAG AA.
- **axe-core** runs in the tests on the empty form, the form with errors and the full page: 0 violations.

7 Payment help assistant (RAG)

A retrieval-augmented assistant over 10 help articles (21 sections) for payment questions.

1. **Retrieve**: BM25 over title, heading and text, with synonym expansion (customers write "OTP", the articles say "code") and light stemming.
2. **Use context**: the decline code of the last payment is added to the question, so "why did it fail?" is answered for *this* payment.
3. **Refuse when unsure**: if the best section scores below 2.25 (chosen on dev questions only), it says it could not find the answer.
4. **Answer with sources**: sentences quoted from the best sections, with the article and section shown. An optional LLM mode must cite the sources it was given, otherwise the quoted answer is used; the model is called through a backend, never with a key in the browser.

Evaluation

Metric (20 held-out test questions)	Result
Right article ranked first	14 / 16 (88%)
Right article in the top 3	15 / 16 (94%)
Out-of-scope questions refused	3 / 4
In-scope questions wrongly refused	0 / 16

> The articles and questions were written by the same person and the test set is small, so these numbers are optimistic. CI fails the build if recall@3 drops below 90%, which catches regressions.

8 Testing and CI/CD

Suite	What it checks
`card.test.ts`	Luhn, network detection incl. range edges, formatting, expiry rules, CVC length
`gateway.test.ts`	Idempotency, lost responses, declines, 3-D Secure, amount validation, no stored card data
`CheckoutForm.test.tsx`	axe checks, autocomplete, error summary focus, double-click = 1 charge, lost response, decline announcement, 3-D Secure, keyboard-only flow
`assistant.test.ts`	Threshold equals the dev-chosen value, recall floor, decline-code context, refusals, LLM citation rules
`App.test.tsx`	Checkout and assistant together; full-page accessibility

GitHub Actions runs all 56 tests and the build on every push; a second workflow deploys the site to GitHub Pages only if the tests pass.

9 Running it

```
npm install
npm run dev        # http://localhost:5173
npm test           # 56 tests
npm run build      # type-check + production build
```

Test cards: `4242 4242 4242 4242` succeeds · `4000 0000 0000 0002` declined · `4000 0000 0000 9995` insufficient funds · `4000 0000 0000 3220` bank verification (code 123456).

10 Limitations and next steps

- The gateway runs in the browser. A real deployment moves it server-side and uses a payment provider's hosted fields, so card data never touches the app (PCI DSS scope).
- The assistant's articles are written for this demo; real help-centre content and real customer questions would give a fairer evaluation.
- Pretrained embeddings would help retrieval with vaguer questions.

11 Interview questions

Why must a retry reuse the same idempotency key?
If the first request charged the card but the response was lost, a new key would create a second charge. The same key makes the gateway return the original result.

Why is the double-click guard a ref?
State updates apply on the next render; two clicks in the same frame would both see "not busy". A ref changes immediately.

What does aria-live do here?

Screen readers announce status changes ("Processing…", "Your card was declined") without the user moving focus.

Why does the assistant refuse some questions?

A confident wrong answer about money is worse than "I could not find this; contact support". The threshold is chosen on dev questions, not test, to avoid overfitting.

What does Luhn not catch?

A valid-looking but non-existent card, and some digit swaps (09 ↔ 90). Only the issuer can confirm a card.